

QEL·QIR·QVM 기반 에이전트-네이티브 컴퓨팅 생태계 토대 설계

Executive summary

QEL의 목표는 또 하나의 범용 프로그래밍 언어를 만드는 것이 아니다. 핵심 목표는 **생성형 모델이 제안한 불확실한 산출물이 권한을 행사하고, 지속 상태를 바꾸며, 다른 에이전트에 사실로 전파되는 전 과정을 타입·효과·증거·검증 규칙 아래 두는 것이다.**

권고하는 토대는 다음과 같다.

QEL → QIR → QVM → QNet

- **QEL**은 인식론적 상태, 권한, 효과, 상태전이, 검증 및 복구를 표현하는 언어다.
- **QIR**은 언어·모델·도구가 생성한 operator를 검증 가능한 공통 중간표현으로 정규화한다.
- **QVM**은 로컬 우선 persistent state, evidence ledger, selective replay, verifier, cognitive JIT 및 deoptimization을 실행한다.
- **QNet**은 개인 기억이 아니라 최소화된 작업 요청, 실행 commitment, 검증 결과, `ExecutionReceipt`, 재사용 가능한 검증 operator를 교환한다.

가장 중요한 설계 결정은 **작고 안정적인 신뢰 코어와, 최대한 자유로운 실험적 dialect를 분리하는 것이다.**

안정 코어

epistemic type
linear capability
declared effect
stage-verify-commit
persistent versioning
receipt provenance
install/deopt boundary

실험적 외곽

causal concept birth
quotient 자동 발견
cognitive JIT
probabilistic verifier
market routing
neural cellular memory
custom hardware lowering

실험적 기능은 얼마든지 추가할 수 있지만, 권한 부여·영구 상태 변경·외부 부작용 commit은 안정 코어를 우회하지 못하게 해야 한다. 이 구조는 MLIR의 확장 가능한 dialect와 단계적 lowering, Move의 resource·ability, Rust의 ownership discipline, eBPF의 설치 전 verifier, Wasm Component Model의 언어 중립 interface contract를 서로 다른 계층에 배치하는 방식이다. ¹

2026년 8월 기준으로 A2A는 Agent Card·Task·Message·Artifact 중심의 에이전트 간 상호운용 계층을 제공하고, 최신 MCP 명세는 tool·resource·prompt와 서버 discovery를 제공한다. SCITT는 2026년 6월 RFC 9943으로 발행되어 signed statement와 transparency receipt의 일반 구조를 제공한다. QEL 생태계는 이 표준들을 대체하기보다, **A2A 위에 증거·권한 semantics를 추가하고, MCP tool을 QEL effect boundary로 감싸며, SCITT를 receipt transparency 계층으로 활용**해야 한다. ²

요청 산출물	토대 결정
QEL 핵심 타입·구문	Proposal → Observation → Attested → Established, affine capability, effect row, operator contract
QIR 명세	typed SSA와 event-transition IR의 혼합, proof obligation과 deopt map 포함
QVM 아키텍처	로컬 persistent state, evidence ledger, verifier plane, Wasm executor, JIT/deoptimizer
네트워크 메시지	최소공개 TaskCapsule, 독립 검증을 기록하는 ExecutionReceipt
실험 체계	성능·언어 안전성·보안·경제·프라이버시를 분리 측정
로드맵	초기 Rust embedded DSL → reference QVM → QNet vertical pilot → MLIR dialect
거버넌스	모델은 operator를 제안할 수 있지만 설치·권한 상승·영구 commit을 스스로 승인할 수 없음

앞선 합성 프로토타입은 이 우선순위를 뒷받침한다. QIR binary는 자연어 표현보다 decode-and-apply가 약 2.06배 빨랐지만 압축 후 크기 이득은 약 1.31배에 그쳤고, 새로운 capability가 생겼을 때 전체 사건의 약 4.98%만 selective replay하여 정답을 복원할 수 있었다. Cognitive JIT는 약 2.29배 빨라졌으나 verifier와 deoptimization이 필수였다. QNet 합성 실험에서는 10% audited-newcomer lane이 기존 surplus의 약 95.7%를 유지하면서 active provider 비율을 약 9.5%에서 25.5%로 높였다. 이 결과는 실제 시장 예측이 아니라 메커니즘 검증용 합성 실험이다. [QVM 선행 실험](#) [QNet 선행 실험](#)

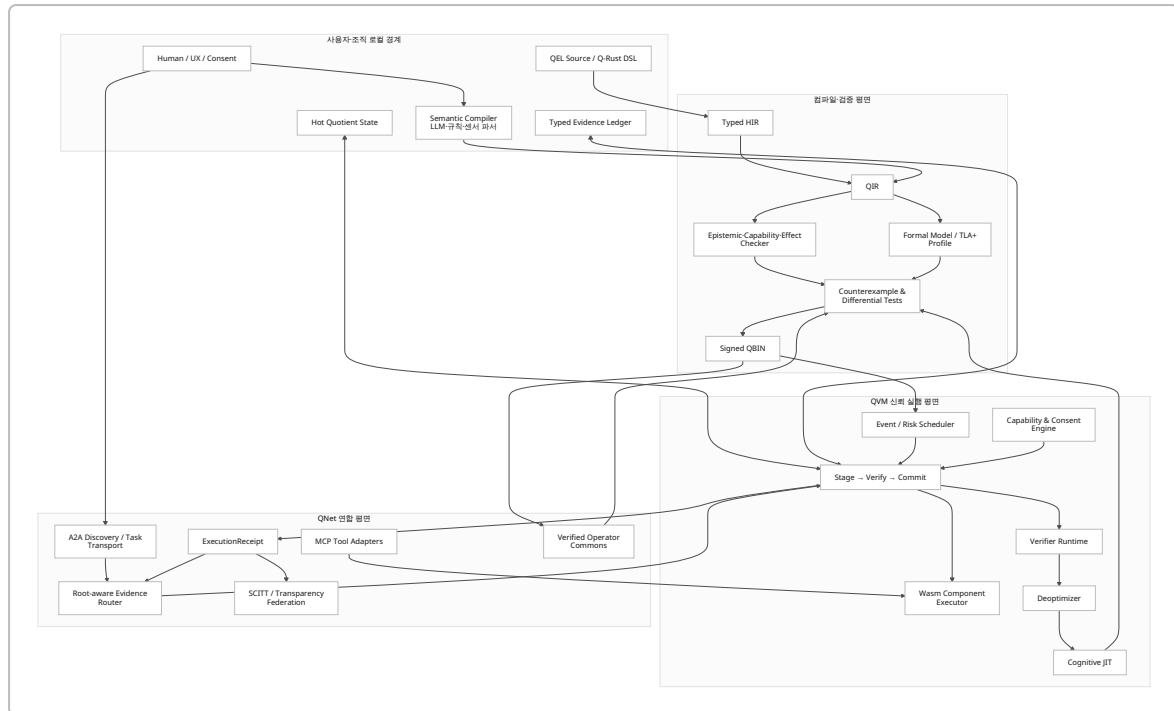
설계 원칙과 전체 아키텍처

QEL 토대는 “새 언어 하나”가 아니라 **언어, 증거 모델, 실행환경, 네트워크 및 거버넌스가 공유하는 의미론**이어야 한다. 이 의미론의 최소 단위는 byte나 함수 호출이 아니라 다음 튜플이다.

$$\mathcal{T} = (S_r, S_w, C, E, G, P, V, K, R, D)$$

- S_r, S_w : 읽고 쓰는 persistent state
- C : 소비·대여·위임되는 capability
- E : 외부 및 내부 effect
- G : 실행 전 guard
- P : 예상 결과와 postcondition
- V : verifier policy와 증거
- K : commit 규칙
- R : rollback 또는 compensation
- D : deoptimization 조건

전체 architecture는 다음과 같다.



안정성 링을 세 단계로 나누는 것이 좋다.

링	포함 요소	변경 정책
신뢰 코어	타입 승격, capability 소비, effect enforcement, transaction, receipt signing	매우 느린 변경, 다중 구현 conformance 필요
표준 dialect	memory projection, network task, policy, verifier, consent	버전별 호환성과 migration 필수
연구 dialect	자동 quotient, causal macrostate, JIT synthesis, NCA memory, 시장 실험	feature gate, sandbox, authority 차단

이 구조는 아이디어 발산을 억제하지 않는다. 오히려 새로운 개념·operator·memory substrate를 dialect로 빠르게 추가하되, 그것이 곧바로 **Established** 사실이나 새로운 권한을 만들지 못하게 한다. MLIR은 서로 다른 추상수준의 dialect를 한 module 안에 공존시키고 conversion pass로 lowering할 수 있으며, custom operation과 type을 ODS로 정의할 수 있다. 따라서 QIR의 장기 backend로 적합하지만, 초기 6개월은 compiler infrastructure 작업이 언어 의미론 검증에 압도하지 않도록 Rust 기반 custom IR로 시작하는 편이 낫다. 3

기존 표준의 책임 경계는 다음처럼 고정한다.

기존 기반	채택할 것	QEL에서 추가할 것
Rust	ownership, borrowing, 안전한 runtime 구현	persistent authority와 epistemic ownership
Move	resource, copy/drop/store ability	목적·시간·횟수·외부 effect가 있는 capability
eBPF	설치 전 verifier, bounded execution 사고방식	semantic safety, evidence, privacy, deopt

기존 기반	채택할 것	QEL에서 추가할 것
Wasm Component/ WIT	portable component와 언어 중립 interface	effect · 권한·검증 behavior
MLIR	extensible dialect와 lowering	QIR semantic operations
Rego	선언적 policy 분리	typed capability와 증거 연계
TLA+	concurrent · distributed invariant 모델링	QEL operator에서 추출한 모델 profile
A2A	agent discovery와 task/artifact transport	root-aware evidence routing
MCP	tool · resource interface	tool effect와 consent boundary
SCITT · Sigstore	signed statement, inclusion proof, append-only transparency	semantic verdict와 verifier independence
VC · RAR · RATS	credential, selective disclosure, 세밀한 권한요청, 원격 attestation	task-bound consent와 실행 receipt

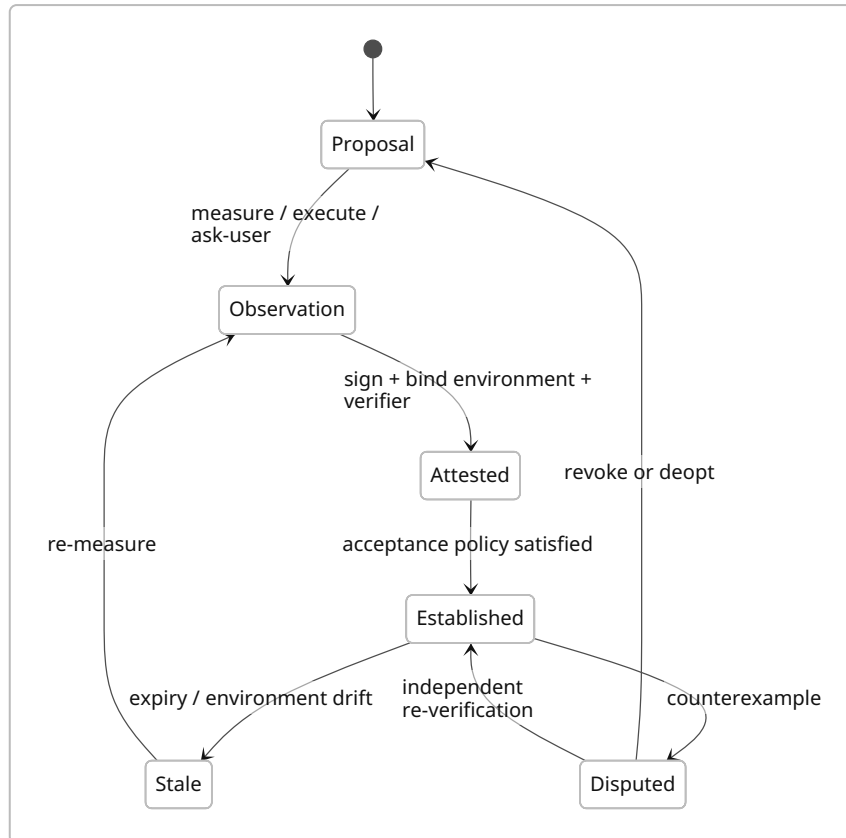
WIT는 component 간 interface contract를 정의하지만 내부 behavior를 정의하지 않는다. Rego는 structured data에 대한 정책 결정을 선언적으로 표현하며, TLA+는 특히 concurrent · distributed system의 상태와 전이를 정밀하게 모델링한다. 따라서 QEL이 실행 semantics를, WIT가 ABI를, Rego profile이 조직 정책을, TLA+ profile이 핵심 invariant를 맡는 분리가 바람직하다. ⁴

QEL 핵심 언어 사양

QEL의 중심은 일반적인 값 타입이 아니라 **근거 수준이 타입에 포함되는 epistemic type**이다.

Proposal<T> ✂ **Observation<T>** ✂ **Attested<T>** ✂ **Established<T>**

이들은 단순 subtype 계층이 아니다. 각 승격에는 명시적인 증거 생성 연산이 필요하다.



권고하는 타입은 다음과 같다.

```

Proposal<T, Source>
Observation<T, Instrument, Time>
Attested<T, VerifierRoot, Environment>
Established<T, Policy, Scope, Expiry>

Disputed<T, Counterexample>
Stale<T, Reason>
Revoked<T, Authority>

```

Established 는 절대적 사실이 아니라 특정 **policy**·**scope**·**expiry** 아래에서 행동 근거로 사용할 수 있는 상태다. 예를 들어 테스트 통과는 특정 commit·환경·시간에만 성립한다.

```

type VerifiedBuild =
    Established<
        BuildArtifact,
        Quorum<2, DistinctControlRoot>,
        RepoCommit,
        Expires<24h>
    >;

```

Move는 `copy`, `drop`, `store`, `key` 같은 ability로 값에 허용되는 bytecode operation을 제한하고, 복제·폐기가 불가능한 resource를 표현한다. Rust의 ownership discipline은 공유 alias를 통한 직접 mutation을 제한하며, RustBelt는 현실적인 Rust subset과 unsafe extension의 안전 조건을 기계적으로 모델링했다. QEL은 이 아이디어를 메모리 안전성에서 **권한 안전성·증거 안전성**으로 확장해야 한다. ⁵

QEL의 typing judgment는 다음 형태가 적합하다.

$$\Gamma; \Delta; \Pi \vdash e : \tau ! \epsilon$$

- Γ : 복제 가능한 일반 값
- Δ : affine 또는 linear capability
- Π : evidence·epistemic context
- ϵ : 선언된 effect 집합

Capability는 세 종류를 기본으로 둔다.

종류	규칙	예시
<code>linear</code>	정확히 한 번 소비	결제 승인, 일회성 배포
<code>affine</code>	최대 한 번 사용, 미사용 폐기 가능	임시 파일 쓰기 권한
<code>shareable</code>	읽기 전용·제한된 복제	공개 데이터 읽기

```
capability Deploy affine {
  resource: ObjectId,
  operation: "git.push",
  branch: "main",
  purpose: PurposeId,
  holder: ControlRoot,
  expires: Time,
  remaining_uses: u8 = 1,
  max_cost: Credits,
}
```

권한 위임은 복사가 아니라 범위 축소다.

```
let child = delegate parent {
  branch = "release/1.2";
  expires = min(parent.expires, now() + 30m);
  remaining_uses = 1;
}
```

QEL의 최소 문법 초안은 다음과 같다.

```
module acme.repo@0.1;

state Repo persistent versioned owned_by ControlRoot {
```

```

    id: ObjectId,
    head: Hash,
    test: TestStatus,
    open_tasks: u32,
}

evidence PytestRun {
    input_root: Hash,
    environment_root: Hash,
    passed: bool,
    failures: list<TestFailure>,
}

operator deploy_patch(
    repo: write Repo,
    patch: Attested<Patch, PytestVerifier, BuildEnv>,
    cap: consume Deploy
) -> Established<Deployment, DeployPolicy, RepoScope, 24h>

reads [repo.head, repo.test]
writes [repo.head, repo.test]
effects [
    fs.read(repo.id),
    sandbox.exec("pytest", timeout = 120s),
    git.write(repo.id),
    network("github.example")
]

requires {
    cap.resource == repo.id;
    cap.purpose == current_task.purpose;
    patch.base == repo.head;
}

stage {
    let candidate: Proposal<Artifact, Compiler> =
        apply(repo.head, patch);

    let run: Observation<PytestRun, Pytest, now> =
        observe pytest(candidate);
}

predicts {
    run.passed;
    candidate.regressions == 0;
}

verify {

```

```

let proof = attest(run)
  by quorum(2, distinct_control_root)
  bound_to [candidate.hash, environment.hash];

establish proof under DeployPolicy;
}

commit {
  repo.head <- candidate.hash;
  emit receipt Deployment;
}

compensate on network.failure {
  restore repo.previous_version;
}

deopt on [
  Counterexample<TestRegression>,
  EnvironmentMismatch,
  PolicyRevoked
];

```

컴파일러는 최소한 다음 프로그램을 거부해야 한다.

금지 패턴	거부 이유
<code>Proposal<T></code> 를 persistent fact에 직접 저장	증거 없는 epistemic 승격
capability 복사	authority 증식
선언되지 않은 network·file·money effect	ambient authority
verifier와 executor의 동일 control-root를 독립 quorum으로 계산	자기검증
irreversible effect를 <code>stage</code> 에서 실행	rollback 불가능
private 값을 <code>declassify</code> 없이 전송	consent·purpose 위반
무증거 <code>merge</code> · <code>split</code>	기억 조작
model-generated operator의 직접 설치	self-authorization
expiry가 지난 <code>Established</code> 값 사용	stale evidence
receipt에 원문 개인정보 삽입	불변 로그의 삭제 불가능성

Quotient는 단순한 데이터 압축 annotation이 아니라 미래 행동·질의 집합에 상대적인 동치관계다.

$$h_1 \sim_{\mathcal{A}, H, \varepsilon} h_2 \iff \forall p \in \mathcal{A}, d(P(Y_{\leq H} \mid h_1, p), P(Y_{\leq H} \mid h_2, p)) \leq \varepsilon$$

```

quotient ActiveRepo over ledger<RepoEvent> {
  action_set: CurrentAgentCapabilities;
}

```



```

horizon: 30d;
tolerance: 0;

distinguish by [
    head,
    test,
    open_tasks,
    active_capabilities,
    unresolved_constraints
];

merge requires EquivalenceCertificate;
split requires
    Counterexample
    | NewCapabilityDependency
    | InvariantViolation;
}

```

`EquivalenceCertificate`에는 최소한 `action_set`, `query_set`, `horizon`, `tolerance`, `event dependencies`, `verifier version`을 넣는다. 새 capability가 추가되면 해당 certificate를 무효화하고, dependency index가 가리키는 evidence만 selective replay한다.

인과적 coarse-graining 연구는 일부 macrostate가 미시상태보다 더 결정적이고 덜 퇴화된 인과 표현을 제공할 수 있다는 가능성을 보여주지만, 최적 coarse-graining과 측정법은 여전히 연구 중이다. 따라서 자동 quotient 발견은 코어 문법이 아니라 `qel.causal` 실험 dialect로 유지하는 것이 안전하다. ⁶

Cognitive JIT 역시 직접 설치가 아니라 다음 승격 pipeline을 거쳐야 한다.

```

primitive trace 수집
→ macro 후보 생성
→ effect·capability 추론
→ primitive interpreter와 differential test
→ counterexample search
→ guard 합성
→ 제한된 canary 설치
→ receipt monitoring
→ promote 또는 deopt

```

최근 counterexample-guided agent 연구에서는 verifier가 반환한 반례가 regex induction에서 학습 효율과 성공률을 높였지만, 이는 범용 agent operator의 안전성을 증명한 결과는 아니다. QEL에서는 해당 접근을 연구적 근거로 삼되, primitive reference와 runtime deopt를 제거해서는 안 된다. ⁷

QIR과 QVM 실행 토대

QIR은 일반 SSA만으로는 부족하다. QEL operator는 값 계산뿐 아니라 persistent state version, effect, evidence, transaction phase, branch, receipt를 표현해야 하기 때문이다. 권고안은 **SSA dataflow와 event-transition graph를 결합한 typed IR**이다.

QIR operation	의미	핵심 검증
<code>q.state.read</code>	특정 version의 state 읽기	read capability
<code>q.state.stage</code>	uncommitted delta 생성	write set · version conflict
<code>q.observe</code>	외부 결과를 Observation으로 생성	instrument · time · input binding
<code>q.attest</code>	Observation에 서명·환경 증거 결합	verifier root · signature
<code>q.establish</code>	policy 충족 시 행동 가능한 사실 생성	policy proof
<code>q.effect.request</code>	외부 effect 요청	declared effect · capability
<code>q.verify</code>	postcondition · oracle · quorum 검사	독립성 · freshness
<code>q.commit</code>	delta와 receipt 원자적 확정	certificate · write conflict
<code>q.compensate</code>	이미 발생한 외부 효과의 보상	compensation contract
<code>q.fork</code> · <code>q.join</code>	counterfactual branch	branch isolation
<code>q.merge</code> · <code>q.split</code>	quotient 갱신	equivalence · counterexample
<code>q.install</code>	새 operator 등록	package signature · tests
<code>q.deopt</code>	macro를 primitive path로 복원	counterexample binding
<code>q.receipt.emit</code>	실행 증거 commitment 생성	privacy schema · signatures

QIR module은 다음과 같이 구성한다.

```

QIR Module
├ semantic type table
├ state schemas and migrations
├ capability types and delegation rules
├ evidence schemas
├ operators
│ ├── read/write sets
│ ├── effect rows
│ ├── stage graph
│ ├── verifier graph
│ ├── commit graph
│ └ compensation/deopt graph
├ quotient projections
├ network message schemas
├ formal invariants
└ provenance and dependency manifest

```

초기 QIR은 Rust enum · arena · typed IDs로 구현한다. 언어 semantics가 안정된 이후 다음과 같은 MLIR dialect로 이동한다.

```
q.state
q.epistemic
q.cap
q.effect
q.tx
q.verify
q.memory
q.net
q.jit
```

MLIR의 dialect conversion은 허용되지 않는 operation을 목표 backend가 지원하는 operation으로 rewrite할 수 있고, LLVM·GPU·SPIR-V 등의 여러 추상수준을 연결한다. QIR에서도 `q.effect.network`를 WASI socket call로 내리거나, `q.verify.hash`를 native·GPU·accelerator implementation으로 내리는 구조를 취할 수 있다. ⁸

Compiler stack은 다음 순서가 현실적이다.

```
Q-Rust macro 또는 .qel
→ Parser
→ Typed HIR
→ Epistemic checker
→ Linear capability checker
→ Effect and privacy checker
→ Transaction-phase checker
→ QIR optimizer
→ Formal profile extraction
→ Static verifier
→ Wasm component + WIT
→ QBIN package
```

`QBIN`은 단순 code binary가 아니라 proof-carrying transition package다.

```
QBIN
├─ Wasm component
├─ WIT world
├─ QIR semantic manifest
├─ state schemas and migrations
├─ capability and effect manifest
├─ verifier policies
├─ formal invariant hashes
├─ differential-test corpus
├─ deoptimization map
├─ SBOM
```

```
└─ provenance root
└─ publisher signature
```

Wasm Component Model은 언어 간 조합을 위해 richer type과 명시적 interface contract를 제공하고, WIT world는 component가 제공하거나 요구하는 interface를 정의한다. 다만 WIT는 behavior를 규정하지 않으므로 effect·evidence·privacy semantics는 QBIN의 QIR manifest가 담당해야 한다. ⁹

QVM runtime은 다음 일곱 subsystem을 최소 구현으로 둔다.

subsystem	책임
Persistent Object Store	content-addressed version, branch, migration
Evidence Ledger	immutable typed event와 provenance
Projection Engine	hot quotient state와 selective replay index
Capability Engine	발급·위임·소비·revocation·consent
Transaction Engine	stage-verify-commit·compensation
Verifier Plane	deterministic oracle, quorum, human approval, attestation
JIT/Deoptimizer	macro promotion, canary, rollback

eBPF verifier는 프로그램이 실행하기 안전한지 load 전에 검사하고, Linux runtime은 verifier 결과를 바탕으로 JIT compilation도 수행한다. 그러나 eBPF 공식 문서는 verifier를 프로그램 의도의 정당성을 판단하는 보안 정책 엔진으로 간주해서는 안 된다고 경고한다. QVM verifier 역시 메모리·종료·effect safety를 검사하는 **정적 verifier**와, 결과의 사실성·독립성·정책 적합성을 판단하는 **semantic verifier**를 분리해야 한다. ¹⁰

Persistent memory는 두 층을 유지한다.

$$\text{Hot State} = f_{\text{projection}}(\text{Relevant Evidence})$$

$$\text{Cold Ledger} = \text{Canonical Typed Evidence}$$

Hot state는 삭제하고 재생성할 수 있어야 한다. Cold ledger는 원본 데이터 전체를 무조건 보존하는 것이 아니라, 향후 재검증에 필요한 typed event, commitment, provenance 및 적절한 보존정책을 따른다. 최근 agent memory system characterization은 여러 시스템에서 query보다 memory construction이 더 큰 비용이 될 수 있고, embedding과 prefill이 write path를 지배할 수 있음을 보고했다. QEL은 모든 사건을 LLM으로 요약·embedding하는 방식을 피하고 deterministic projection을 우선해야 한다. ¹¹

Neural Cellular Automata 기반의 분산·자가복구 기억은 장기 연구 dialect로 유지할 가치가 있다. 최근 연구는 local interaction으로 오류를 교정하는 다양한 비평형 기억 동역학을 학습할 수 있음을 보였지만, 복잡한 symbolic binding이나 provenance 보존을 해결한 것은 아니다. 따라서 NCA는 identity invariant나 redundancy layer 실험에는 적합하지만, Evidence Ledger의 대체물로 사용해서는 안 된다. ¹²

네트워크·프라이버시·경제 생태계

QNet의 원칙은 개인 기억은 로컬에, 검증 가능한 전이만 네트워크에다.

네트워크 객체는 세 종류로 제한한다.

Advertisement

후보 탐색을 위한 비신뢰 자기소개

TaskCapsule

목적·예산·요구 capability·최소공개 predicate

ExecutionReceipt

실행·검증·환경·commitment·정산 결과

TaskCapsule 예시는 다음과 같다.

```
{
  "type": "qnet.task-capsule.v0",
  "task_id": "urn:qtask:7fa2...",
  "requester_root": "did:qroot:user-41",
  "purpose": {
    "id": "fix-tests",
    "human_summary": "지정 commit의 실패 테스트 수정"
  },
  "inputs": [
    {
      "kind": "repository-snapshot",
      "commitment": "sha256:ab91...",
      "delivery": "encrypted-after-match"
    }
  ],
  "required_capabilities": [
    "python.pytest",
    "git.patch",
    "sandbox.no-network"
  ],
  "authorization": {
    "resource": "repo:81",
    "operations": ["read", "propose-patch"],
    "expires_at": "2026-08-05T16:00:00+09:00",
    "max_uses": 1
  },
  "disclosure": {
    "predicates": {
      "language": "python",
      "test_framework": "pytest"
    },
    "forbidden": ["user_memory", "unrelated_files"]
  },
  "settlement": {
    "budget": {"currency": "KRW", "max": 50000},
```

```

"verifier_policy": {
  "kind": "quorum",
  "count": 2,
  "distinct_control_roots": true
}
}
}

```

ExecutionReceipt 는 다음과 같다.

```

{
  "type": "qnet.execution-receipt.v0",
  "task_id": "urn:qtask:7fa2...",
  "requester_root": "did:qroot:user-41",
  "executor_root": "did:qroot:provider-9",
  "verifier_roots": [
    "did:qroot:verifier-3",
    "did:qroot:verifier-8"
  ],
  "operator": {
    "qbin": "sha256:ff20...",
    "source": "qel:repo.fix-pytest@0.3.1"
  },
  "commitments": {
    "input": "sha256:ab91...",
    "output": "sha256:cd04...",
    "environment": "sha256:90d3..."
  },
  "attestation": {
    "rats_evidence": "cose:...",
    "runtime": "qvm-0.2"
  },
  "verdict": {
    "result": "accepted",
    "policy": "pytest-reproduce-2of3",
    "measurements": {
      "passed": 184,
      "failed": 0
    }
  },
  "privacy": {
    "disclosed_fields": 4,
    "personal_data_on_log": false
  },
  "transparency": {
    "service": "scitt:qnet-cell-kr1",
    "receipt": "cose:...",

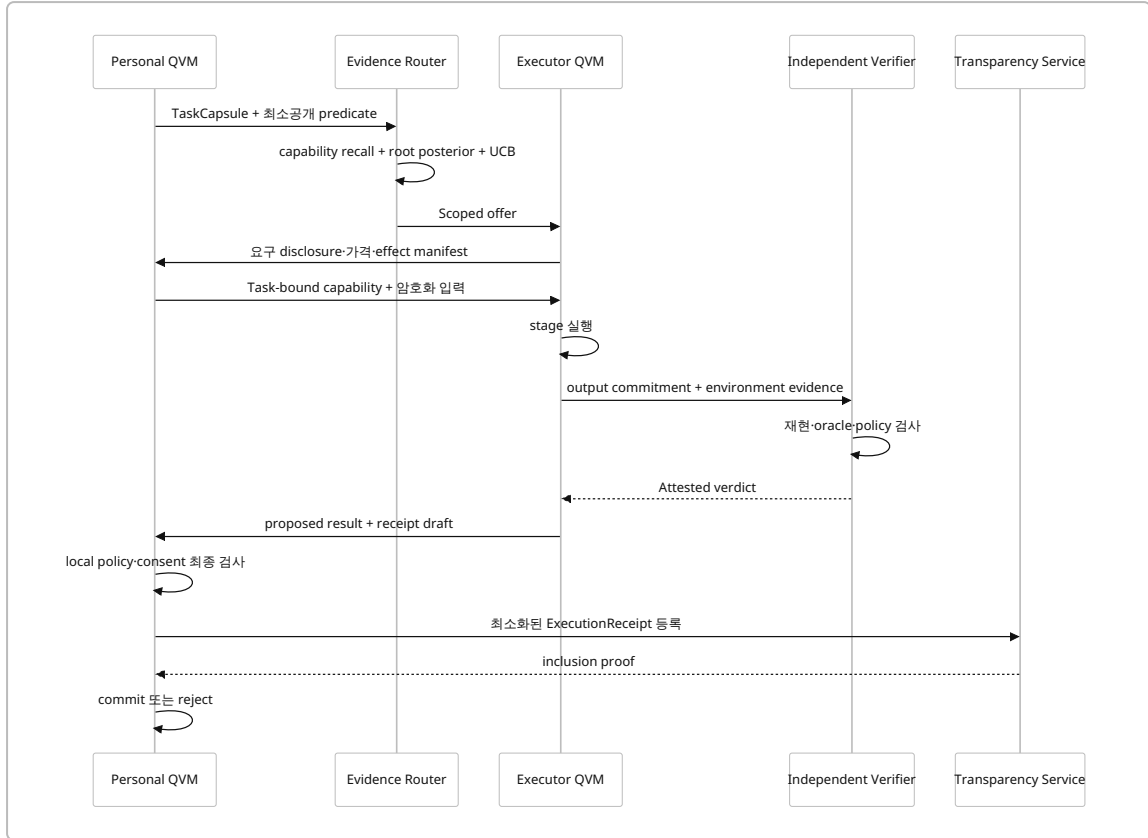
```

```

"inclusion_proof": "merkle:..."
},
"signatures": ["cose:executor...", "cose:verifier..."]
}

```

실행 흐름은 다음과 같다.



SCITT의 transparency service는 signed statement를 등록하고 receipt와 검증 가능한 데이터 구조를 통해 투명성을 제공하지만, 어떤 issuer를 신뢰할지 결정하는 문제는 relying party의 영역이다. Sigstore Rekord도 artifact signature의 append-only 기록과 inclusion·integrity verification을 제공한다. 따라서 “로그에 기록됐다”는 사실을 “작업이 참되게 수행됐다”와 동일시해서는 안 된다. ¹³

Routing은 전역 scalar reputation이 아니라 capability·task type·environment·outcome별 posterior를 사용한다.

$$\begin{aligned}
 Score(r, t) = & \mathbb{E}[Success \mid r, c_t, o_t, e_t] \\
 & + \alpha UCB(r, c_t) - \beta Cost(r, t) - \gamma Latency(r, t) \\
 & - \delta Concentration(r, c_t) - \rho PrivacyExposure(r, t) + \eta NewcomerEligibility(r, c_t)
 \end{aligned}$$

핵심 규칙은 다음과 같다.

- Advertisement는 recall에만 사용하고 순위는 receipt evidence로 결정한다.
- requester, executor, verifier의 독립성은 identity 문자열이 아니라 `control-root` 기준으로 계산한다.
- 동일 root가 만든 여러 alias는 verifier quorum과 newcomer slot을 늘리지 못한다.
- 신규 provider에게는 저위험·고검증 task의 제한된 newcomer lane을 제공한다.

- 과도한 집중이 생기면 UCB, capacity-aware routing, 수익 concentration penalty를 적용한다.
- subjective task는 deterministic task와 같은 pass/fail reputation pool에 넣지 않는다.

Sybil 공격에 관한 고전적 결과는 일반적인 분산 환경에서 신뢰할 수 있는 identity anchor 없이 여러 가짜 identity를 완전히 방지하기 어렵다는 점을 보였다. 따라서 `control-root`는 완전한 proof-of-personhood가 아니라 device attestation, organization credential, payment relationship, passkey, recovery graph 등을 조합한 **assurance-graded control boundary**여야 한다. ¹⁴

```
type ControlRoot = Root<
  assurance: {
    device: RATSLevel,
    organization: Option<VC>,
    payment: Option<AccountBinding>,
    human: Option<Credential>
  }
>;
```

RATS는 attester가 생성한 evidence를 verifier가 평가하고 relying party가 trust decision에 활용하는 architecture를 정의한다. VC 2.0은 issuer·holder·verifier 모델과 selective disclosure를 지원하며, OAuth RAR는 구조화된 세밀한 authorization detail을 전달한다. 이 세 가지를 QEL capability와 결합하면 “누구인가”보다 “어떤 환경에서, 어떤 task를 위해, 어느 범위의 권한을 행사할 수 있는가”를 표현할 수 있다. ¹⁵

프라이버시는 네 개의 data plane으로 나눈다.

plane	내용	기본 정책
Private State	기억·목표·파일·관계	장치·조직 내부
Selective Predicate	task matching에 필요한 조건	최소공개·expiry
Encrypted Artifact	실제 입력과 출력	매칭·승인 후 제한 제공
Public Receipt	commitment·역할·판정	개인정보 원문 금지

Consent도 UI checkbox가 아니라 versioned capability로 표현한다.

```
capability Consent affine {
  subject: UserRoot,
  controller: ServiceRoot,
  purpose: PurposeId,
  data_classes: set<DataClass>,
  recipients: set<ControlRoot>,
  expires: Time,
  revocable: true,
  onward_transfer: false,
}
```

Append-only transparency log에 개인정보 원문을 기록하면 철회·삭제 요구와 충돌할 수 있다. 따라서 공개 ledger에는 commitment와 비개인적 metadata만 두고, 실제 데이터는 삭제 가능한 private storage에 보관하며,

필요하면 암호키 폐기로 접근을 제거해야 한다. NIST는 data minimization이 무단 접근·사용에 노출되는 개인정보를 줄인다고 설명하고, 한국 개인정보보호위원회의 생성형 AI 안내서는 AI 수명주기 전반의 개인정보 처리와 안전조치를 요구한다. ¹⁶

경제 primitive는 토큰 없이 시작한다.

```
Requester payment
├─ Executor fee
├─ Verifier fee
├─ Operator royalty
├─ Challenge reserve
└─ Infrastructure fee
```

- 결제는 법정화폐, 조직 credit 또는 invoice로 처리한다.
- deterministic verifier는 고정 수수료를 받는다.
- 고비용 재현에는 refundable challenge bond를 사용할 수 있다.
- operator 저자는 실제 사용과 성공 receipt에 따라 royalty를 받는다.
- self-task, self-execution, self-verification은 보상 대상에서 제외한다.
- reputation 자체를 판매하거나 양도할 수 없게 한다.
- 신규자 지원은 영구 보조가 아니라 첫 독립 receipt를 얻기 위한 검증비 보조로 제한한다.

초기 UX는 범용 agent marketplace보다 **objective vertical cell**로 시작한다. 추천 순서는 software build/test, 데이터 변환·검증, 규칙 기반 document compliance, simulation benchmark다. 한국 KISA도 SW 공급망에서 SBOM, 구성요소 추적성, 취약점 관리와 공급망 투명성을 강조하고 있어, QBIN·ExecutionReceipt·operator provenance를 검증할 초기 vertical로 소프트웨어 공급망이 적합하다. ¹⁷

실험·시뮬레이션과 평가 체계

평가 체계는 모델 성능 하나로 축약하지 않는다. 최소한 언어 안전성, 실행 성능, 메모리 적응성, 네트워크 보안, 경제적 건강성, 프라이버시를 독립적으로 측정해야 한다.

실험군	주요 매개변수	핵심 가정	측정지표	초기 go/no-go 기준
Epistemic typing	불법 승격 pattern 100–10,000 개	모델은 악의적·실수 가능	compile rejection, false reject	직접 Proposal→Established 우회 0건
Capability safety	delegation depth 1–20, alias 1–64	alias가 권한을 늘려 함	authority amplification, use-after-revoke	권한 총량 증가 0, revoke 후 effect 0
Transaction	crash probability 0–20%, external failure	process·network는 실패	atomicity, duplicate effect, recovery latency	state divergence 0, irreversible duplicate 0

실험군	주요 매개변수	핵심 가정	측정지표	초기 go/no-go 기준
Selective replay	event 10^5 – 10^8 , dependency selectivity 0.1–100%	capability set이 변화	replay fraction, answer equivalence	declared query 100% 동치, median replay <10%
Quotient	merge ratio, horizon, tolerance	잘못된 merge가 가능	state count, regret, split latency	counterexample 후 bounded split·정확 복원
Cognitive JIT	macro length 2–100, drift 0–20%	반복 pattern과 regime change 존재	speedup, probe count, deopt loss	1.5배 이상 및 deopt 후 exact equivalence
Verifier	oracle 오류 0–30%, collusion 0–50%	검증자도 실패·담합	false accept/reject, independence	root-aware가 identity-aware보다 유의하게 우수
Sybil	malicious roots 0–30%, aliases 1–64	identity 생성 저비용	routing capture, false acceptance	alias 증가에 quorum 영향 없음
Concentration	공급자 20–10,000, task skew	evidence가 incumbent에 축적	Gini, top-10% share, active provider	surplus 95% 유지하며 participation 2배
Newcomer lane	탐색률 0–30%	신규자는 receipt 없음	first-receipt latency, surplus	4–10% 범위에서 Pareto frontier 확인
Privacy	공개 field 0–100, 재식별 공격	metadata 결합 가능	fields/task, mutual information, incidents	원문 PII ledger 기록 0
Subjective task	objectivity 0.1–0.9	판정이 사용자별로 다름	calibration, disagreement, appeal	objective pool과 분리 시 개선 확인
Scalability	node 20–100k, receipt 10^3 – 10^9	연합 log와 index가 성장	p50/p99 latency, bytes/task	선형 full scan 없이 routing

권고 benchmark는 세 묶음이다.

benchmark	목적	데이터
QEL - Conform	타입·권한·effect·transaction conformance	수작업 adversarial program, property-based generated program
QVM-Bench	state, replay, crash, JIT, deopt	synthetic event stream와 실제 coding-agent trace
QNet-Sim	routing, Sybil, concentration, privacy, settlement	agent pool simulator와 점진적 실제 vertical trace

핵심 metric은 다음과 같다.

$$SemanticPageFaultRate = \frac{\text{기존 operator로 처리할 수 없는 event}}{\text{전체 event}}$$

$$ReplaySelectivity = \frac{\text{새 capability에 재생한 evidence}}{\text{전체 evidence}}$$

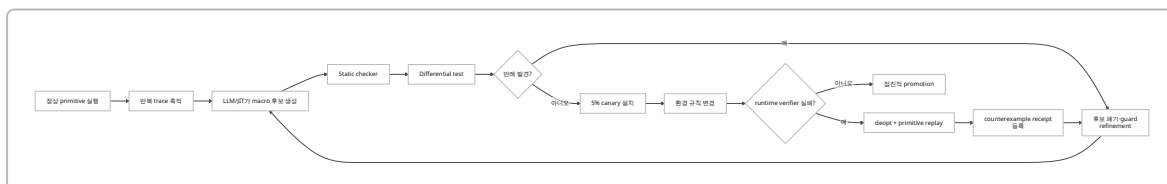
$$VerifiedCoverage = \sum_q P_D(q) \mathbf{1}[\exists r : covers(r, q) \wedge evidence(r, q) \geq \tau]$$

$$IndependentFalseAccept = \frac{\text{독립 검증을 통과한 잘못된 결과}}{\text{외부 accepted task}}$$

$$DeoptRegret = U(\text{oracle execution}) - U(\text{JIT then deopt})$$

$$PrivacyCost = w_f \cdot disclosedFields + w_i \cdot inferredAttributes + w_r \cdot retentionTime$$

테스트 scenario는 다음처럼 구성한다.



데이터는 다음 우선순위로 확보한다.

우선순위	데이터	용도
최우선	synthetic typed event generator	edge case·규모·ground truth
높음	GitHub issue-patch-test-review trace	operator·verifier·receipt
높음	CI/CD, SBOM, build provenance	deterministic vertical
중간	MCP tool-call trace	effect inference와 consent
중간	A2A task lifecycle capture	network state machine
중간	사용자 승인·취소·수정 로그	consent UX
연구	simulation·robotics trace	physical irreversible effect
연구	subjective panel data	비결정적 settlement

실제 사용자 데이터는 원문을 중앙 수집하기보다 로컬에서 typed event로 변환한 후 익명화·commitment·집계 metric만 제공하는 구조가 바람직하다. 학술 공개 데이터는 재현성을 위해 사용하되, personal agent trace는 별도 consent와 삭제 가능한 저장소를 요구한다.

통계 분석은 평균만 보고 결론 내리지 않는다. seed, task distribution, malicious fraction, verifier capacity를 계층화하고 bootstrap confidence interval, ablation, adversarial worst case, Pareto frontier를 보고한다. 특히 surplus가 좋아졌더라도 false acceptance, concentration, privacy cost가 악화되면 성공으로 판정하지 않는다.

프로토타입 로드맵과 구현 우선순위

권고 baseline team은 8-10명이다.

Programming languages / compiler	2
Rust runtime / storage	2
Security / distributed systems	2
Agent / verifier / JIT	1-2
Simulation / data	1
Product / consent UX	1
Formal methods	0.5-1

12개월 계획은 다음과 같다.

기간	우 선 순 위	구현 내용	주요 인력	exit milestone
1-2 개월	P0	QEL semantic core, type lattice, operator grammar, threat model	PL·security·formal	30개 대표 program과 100개 불법 program 명세
2-3 개월	P0	Q-Rust macro, typed HIR, epistemic·capability checker	PL·Rust	직접 승격·cap 복제·미선언 effect 정적 차단
3-4 개월	P0	QIR reference interpreter, persistent object, event ledger	Runtime·PL	deterministic replay와 version migration
4-5 개월	P0	stage-verify-commit, crash recovery, compensation	Runtime·security	fault injection에서 state divergence 0
5-6 개월	P0	Wasm/WIT executor, MCP effect wrapper, QBIN package	Runtime·agent	세 개 독립 언어 component 실행
6-7 개월	P1	selective replay, quotient certificate, capability shock	Memory·data	새로운 query에서 replay selectivity 측정
7-8 개월	P1	verifier framework, differential testing, JIT/deopt	Agent·formal	의도적 macro fault 탐지와 exact deopt

기간	우 선 순 위	구현 내용	주요 인력	exit milestone
8-9 개월	P1	TaskCapsule, ExecutionReceipt, A2A adapter, SCITT test log	Distributed · security	5-node end-to-end receipt
9-10 개월	P1	evidence router, root model, newcomer lane, simulator	Simulation · distributed	Sybil · 집중 ablation report
10- 11개 월	P1	software build/test vertical pilot, consent UX	Product · agent · security	2-3 design partner, objective oracle
11- 12개 월	P1	conformance suite, red-team, governance process, alpha SDK	전원	QEL 0.1, QVM 0.1, QNet profile 0.1
이후	P2	MLIR dialect, formal proof 확대, hardware profiling, NCA 연구	Compiler · research	opcode profile이 실제 workload에서 안정될 때만

구현 checklist는 다음과 같이 우선순위를 고정한다.

등 급	구현 항목	이유	완료 기준
P0	Epistemic type checker	환각과 사실의 구조적 분리	승격은 proof-producing function만 가능
P0	Affine/linear capability	self-escalation 방지	alias · serialization 후에도 보존
P0	Effect manifest	ambient authority 제거	runtime host call과 정적 manifest 일치
P0	Transaction phase checker	rollback 불가능 effect 통 제	irreversible effect는 승인된 commit phase만
P0	Persistent versioned state	durable agent identity	crash · migration 후 동일 root
P0	Typed evidence ledger	selective replay와 provenance	projection 삭제 후 재생성
P0	Reference verifier	의미론 기준점	optimized QVM과 differential test
P0	Revocation · expiry	장기 agent의 stale authority 방지	revoke 후 effect 0
P1	Quotient certificate	상태 압축의 명시적 가정	merge/split provenance
P1	Cognitive JIT/deopt	속련과 성능	guard · counterexample · rollback
P1	ExecutionReceipt	네트워크 신뢰 데이터	distinct-root 검증
P1	Evidence router	검증 기반 discovery	scalar reputation 금지
P1	Newcomer lane	incumbent lock-in 완화	first-receipt latency 감소

등급	구현 항목	이유	완료 기준
P1	Consent capability	purpose-bound disclosure	UI와 runtime 동일 객체
P1	TLA+ extraction	distributed invariant 검증	핵심 transaction model check
P2	MLIR QIR dialect	backend 확장	semantics 안정 후 진행
P2	Operator marketplace	ecosystem 성장	provenance·challenge 완성 후
P2	Neural/topological memory	자가복구 연구	ledger와 분리된 실험
P2	Custom ISA·hardware	병목 가속	동일 QIR op가 다수 workload 지배 시

아이디어 발산을 최대화하려면 QEP-QEL Enhancement Proposal 절차를 둔다.

아이디어

- Experimental dialect
- threat model
- reference interpreter semantics
- negative tests
- capability/effect boundary
- opt-in feature flag
- pilot evidence
- standard dialect 후보
- core 편입 여부 별도 심사

Core 편입 기준은 “흥미롭다”가 아니라 다음이어야 한다.

- 두 개 이상의 독립 구현
- machine-readable conformance suite
- authority amplification이 없음
- rollback·deopt path 존재
- privacy impact 분석
- 실제 workload에서 반복되는 필요
- 이전 semantics의 migration 가능
- 형식 모델 또는 명시적 비정형 가정

OS·ISA는 12개월 범위에서 제외한다. 먼저 userspace QVM에서 QIR opcode frequency, serialization, state projection, verifier latency, effect mediation이 실제 병목인지 측정한다. 병목이 stable하지 않은 상태에서 hardware semantics를 고정하면 아이디어 발산을 오히려 막는다.

실패 조건·윤리·거버넌스·참조 우선순위

프로젝트는 다음 조건 중 하나가 반복되면 핵심 가설을 수정하거나 폐기해야 한다.

실패 조건	판정
대부분의 operator가 결국 unrestricted Python·shell escape를 요구	QEL effect model이 현실을 포착하지 못함
capability와 effect annotation이 실제 코드보다 더 복잡	언어 abstraction 실패
새 capability마다 ledger 대부분을 replay	dependency model과 quotient 실패
state 수가 event 수에 비례해 증가	의미적 압축 실패
verifier 비용이 실행 비용을 지속적으로 초과	검증 경제성 실패
JIT macro가 잦은 drift로 대부분 deopt	operator 안정성 부족
receipt가 결과 진실성보다 서명 형식만 측정	network evidence 실패
root assurance가 alias를 유의미하게 묶지 못함	Sybil 방어 실패
newcomer 지원 없이 공급자가 극소수에 고착	ecosystem 지속 가능성 실패
newcomer lane이 공격자 보조금으로 변함	탐색 정책 실패
immutable log에 삭제 불가능한 개인정보 축적	privacy architecture 실패
사용자가 consent와 capability 범위를 이해하지 못함	UX·자율성 실패
subjective task가 false confidence를 생성	settlement model 실패
모델이 verifier prompt나 policy를 조작	trust boundary 실패
deopt가 외부의 비가역적 피해를 복구하지 못함	physical·financial domain 부적합

안전·윤리 checklist는 배포 gate로 운용한다.

영역	필수 질문
인간 통제	고위험 effect에 취소·승인·appeal 경로가 있는가
인식론	추측·관측·증명·정책상 수용이 UI와 receipt에서 구분되는가
권한	모델이 새로운 capability를 생성하거나 범위를 확대할 수 없는가
프라이버시	목적 외 데이터와 불변 로그의 개인정보가 없는가
동의	consent가 구체적 목적·수신자·기간·effect에 묶이는가
공정성	newcomer가 독립 receipt를 획득할 현실적 경로가 있는가
집중	수익·routing·검증 권력이 특정 root에 고착되는가
검증 독립성	requester·executor·verifier가 실질적으로 분리되는가
복구	rollback 불가능 effect에 compensation·보험·human review가 있는가
설명	사용자는 누가 무엇을 왜 실행했는지 receipt로 이해할 수 있는가
이의제기	잘못된 receipt·평판·revocation에 대한 정정 절차가 있는가
공급망	QBIN·operator·verifier의 SBOM·signature·provenance가 있는가

거버넌스는 세 권력을 분리한다.

조직	권한	금지
Language TSC	문법·타입·QIR·conformance	개별 operator 승인
Safety & Semantics Board	core invariant, 위험 profile, emergency revoke	경제적 routing 운영
Ecosystem Council	receipt profile, routing, newcomer·fee 정책	언어 안전 규칙 완화

모델 개발사, marketplace 운영자, verifier 사업자가 동일 조직이더라도 governance vote에서는 독립 actor로 계산해서는 안 된다. Emergency revocation은 가능해야 하지만 사후 공개 사유, 범위, 기간, appeal을 요구한다. Core semantic 변경은 버전별 replay 가능성과 migration proof를 동반해야 한다.

한국에서는 인공지능기본법과 시행령이 2026년 현재 시행 중이며, 생성형·고영향 AI 기반 서비스의 사전 고지와 생성물 표시 등 투명성 의무를 규정한다. QEL receipt와 UX는 AI 사용 사실, 생성 산출물, 실행 주체, 검증 수준을 기계와 인간 모두가 확인할 수 있게 설계해야 한다. ¹⁸

NIST AI RMF는 AI를 설계·개발·배포·사용하는 조직이 신뢰성 위험을 관리하도록 지원하며, Generative AI Profile은 생성형 AI 특유의 위험과 대응을 보완한다. QEL release마다 Govern-Map-Measure-Manage에 대응하는 safety case를 만들고, 언어 수준 보장과 운영 수준 위험을 구분하는 것이 적절하다. ¹⁹

포함 우선 소스 목록은 다음 순서로 유지한다.

등급	우선 참조
원전 표준	MLIR 언어·dialect·conversion 문서, Move Book, RustBelt, eBPF verifier, Wasm Component/WIT, A2A v1, MCP 최신 명세, RFC 9943 SCITT, Sigstore Rekor, RFC 9334 RATS, RFC 9396 RAR, W3C VC 2.0, OPA/Rego, TLA+
학술 연구	causal emergence·coarse-graining, verifier-guided learning, agent memory systems characterization, NCA robust memory
국내 원문	국가법령정보센터 인공지능기본법·시행령, 개인정보보호위원회 생성형 AI 개인정보 안내서, KISA SW 공급망·SBOM 자료
보조 자료	실제 구현 보고서, 산업 사례, benchmark 및 공개 코드
후순위	마케팅 문서, 출처 없는 architecture blog, 자체 평가만 있는 vendor benchmark

최종 토대 명제는 다음과 같다.

QEL의 핵심은 AI가 더 많은 프로그램을 쓰게 하는 것이 아니라,

AI가 만든 제안이 언제 사실\cdotp권한\cdotp행동\cdotp기억이 될 수 있는지를 계산 가능한 계약으로 만드는

아이디어 발산을 최대화하려면 언어의 표현력만 늘려서는 안 된다. 실험적 아이디어가 안전한 경계 안에서 빠르게 설치되고, 반례가 나오면 국소적으로 철회되며, 권한과 영구 상태는 검증된 경로로만 변경되는 기반을 먼저 세워야 한다.

- 1 <https://mlir.llvm.org/docs/LangRef/>
<https://mlir.llvm.org/docs/LangRef/>
- 2 <https://a2a-protocol.org/latest/specification/>
<https://a2a-protocol.org/latest/specification/>
- 3 <https://mlir.llvm.org/docs/Tutorials/CreatingADialect/>
<https://mlir.llvm.org/docs/Tutorials/CreatingADialect/>
- 4 <https://component-model.bytecodealliance.org/design/wit.html>
<https://component-model.bytecodealliance.org/design/wit.html>
- 5 <https://move-language.github.io/move/abilities.html>
<https://move-language.github.io/move/abilities.html>
- 6 <https://arxiv.org/abs/2201.10154>
<https://arxiv.org/abs/2201.10154>
- 7 <https://arxiv.org/abs/2606.11521>
<https://arxiv.org/abs/2606.11521>
- 8 <https://mlir.llvm.org/docs/DialectConversion/>
<https://mlir.llvm.org/docs/DialectConversion/>
- 9 <https://component-model.bytecodealliance.org/design/why-component-model.html>
<https://component-model.bytecodealliance.org/design/why-component-model.html>
- 10 <https://docs.ebpf.io/linux/concepts/verifier/>
<https://docs.ebpf.io/linux/concepts/verifier/>
- 11 <https://arxiv.org/html/2606.06448v1>
<https://arxiv.org/html/2606.06448v1>
- 12 <https://arxiv.org/html/2508.15726v1>
<https://arxiv.org/html/2508.15726v1>
- 13 <https://datatracker.ietf.org/doc/rfc9943/>
<https://datatracker.ietf.org/doc/rfc9943/>
- 14 https://link.springer.com/chapter/10.1007/3-540-45748-8_24
https://link.springer.com/chapter/10.1007/3-540-45748-8_24
- 15 <https://datatracker.ietf.org/doc/rfc9334/>
<https://datatracker.ietf.org/doc/rfc9334/>
- 16 <https://pages.nist.gov/800-63-4/sp800-63a/privacy/>
<https://pages.nist.gov/800-63-4/sp800-63a/privacy/>
- 17 <https://www.kisa.or.kr/2060204/form?page=1&postSeq=15>
<https://www.kisa.or.kr/2060204/form?page=1&postSeq=15>
- 18 <https://www.law.go.kr/lsInfoP.do?ancYnChk=0&lsId=014820>
<https://www.law.go.kr/lsInfoP.do?ancYnChk=0&lsId=014820>
- 19 <https://www.nist.gov/itl/ai-risk-management-framework>
<https://www.nist.gov/itl/ai-risk-management-framework>